```
death').verbose _ name
•         self.assertEquals(field _ label, 'died')
•
•     def test _ first _ name _ max _ length(self):
•         author = Author.objects.get(id=1)
•         max _ length = author. _ meta.get _ field('first _
name').max _ length
•         self.assertEquals(max _ length, 100)
•
•     def test _ object _ name _ is _ last _ name _ comma _
first _ name(self):
•         author = Author.objects.get(id=1)
•         expected _ object _ name = f'{author.last _ name},
{author.first _ name}'
•         self.assertEquals(expected _ object _ name,
str(author))
•
•     def test _ get _ absolute _ url(self):
•         author = Author.objects.get(id=1)
•         # This will also fail if the urlconf is not
defined.
•         self.assertEquals(author.get _ absolute _ url(), '/
catalog/author/1')
```

Here, we have imported TestCase to get the AuthorModelTest. We have used a descriptive name to identify the failing tests. Next, we use setUpTestData() to create an author object. Next, you can execute the following code for values of the field labels, and character size.

```
• # Get an author object to test
• author = Author.objects.get(id=1)
•
• # Get the metadata for the required field and use it to
query the required field data
• field _ label = author. _ meta.get _ field('first _ name').ver-
bose _ name
•
• # Compare the value to the expected result
• self.assertEquals(field _ label, 'first name')
```

Next you can execute custom test methods for last name, and first name.